ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

Module 6: Containerization with Docker

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Foundations of Containerization

- Why **containers** appeared in modern software deployment
- The classic problem: **"it works on my machine"**
- How containers help DevOps teams ship faster and safer

The "It Works on My Machine" Problem

- Developers often say: “The app works on my laptop but crashes on the server”
- Causes:
  - Different OS versions and installed libraries
  - Different runtime versions (Node.js, Java, .NET, Python, etc.)
  - Missing system packages or tools
- This leads to:
  - Wasted debugging time
  - Friction between **dev** and **ops** teams
  - Slow, unreliable deployments

What Is a Container?

- A **container** is a lightweight, isolated environment for running applications
- Bundles:
  - Application code
  - Runtime and libraries
  - System tools that the app needs
- Shares the **host OS kernel**, but isolates:
  - Filesystem
  - Processes
  - Network interfaces
- Think of it as a **lightweight VM for one application**

Containers vs Virtual Machines (Conceptual)

- **Virtual Machine (VM)**:
  - Emulates hardware and runs a full OS
  - Heavy: each VM has its own OS kernel
  - Slower to boot, consumes more RAM and disk
- **Container**:
  - Shares host OS kernel
  - Only includes app + dependencies
  - Fast to start, lower resource overhead
- DevOps impact:
  - More containers per host
  - Faster scaling and rollbacks

Containers vs VMs (Practical View)

- VMs are great for:
  - Strong isolation between tenants
  - Running different OS types on the same host
- Containers are great for:
  - High-density microservices
  - Rapid deployment and scaling
  - CI/CD pipelines and ephemeral environments
- In practice, we often run containers **inside** VMs or cloud instances

Real-World Benefits of Containers

- **Portability**:
  - Same container image runs on your laptop, CI server, or cloud
- **Reproducibility**:
  - Environment is defined in code (Dockerfile)
- **Scalability**:
  - Quickly start more containers when traffic increases
- **Automation**:
  - Works well with CI/CD tools (GitHub Actions, GitLab CI, Jenkins, etc.)

Containers in DevOps Workflows

- Typical workflow:
  - Developer builds a **Docker image** locally
  - CI pipeline runs tests inside containers
  - Image is pushed to a **registry**
  - Deployment tool (e.g. Kubernetes, ECS) pulls the image and runs containers
- Result:
  - Fewer “works on my machine” incidents
  - Clear separation between build and run stages

Core Docker Terminology & Concepts

- **Image, container, layer, registry**
- **Port mapping**: `-p host:container`
- **Volumes**: bind mounts vs named volumes
- **Networks**: bridge, host, custom networks
- Container **lifecycle**

Image vs Container

- **Docker image**:
  - Read-only template
  - Contains app, dependencies, and configuration
  - Built from a **Dockerfile**
- **Docker container**:
  - Running (or stopped) instance of an image
  - Has its own writable layer
  - You can start, stop, and delete it without affecting the image

Layers in Docker Images

- Docker images are composed of **layers**
- Each instruction in a Dockerfile (e.g. `RUN`, `COPY`) creates a new layer
- Layers can be reused between images:
  - Speeds up builds
  - Saves disk space
- Example:
  - Base OS layer
  - Runtime layer (Node.js)
  - Application code layer

Docker Registries

- **Registry**: server that stores and distributes Docker images
- Common registries:
  - **Docker Hub** (public & private repos)
  - **AWS Elastic Container Registry (ECR)**
  - GitHub Container Registry
- Workflow:
  - `docker push` to upload images
  - `docker pull` to download images on other machines

Port Mapping: Exposing Container Services

- Containers have their own **internal network**
- To access a service from the host, expose ports:
  - `-p hostPort:containerPort`
- Example:
  - Container runs a web server on `8080`
  - Map it to host port `80`:

    ```
    docker run -p 80:8080 my-web-image
    ```

- Now users can access it via `http://localhost:80`

Volumes: Persistent Data

- Containers are **ephemeral**: if you delete a container, its internal data is gone
- Use **volumes** to persist data:
  - **Bind mount**: map a host directory into the container
  - **Named volume**: managed by Docker, stored under Docker's data directory
- Useful for:
  - Databases (MongoDB, MySQL)
  - Logs
  - Uploaded files

Volumes: Bind Mounts vs Named Volumes

- **Bind mount**:
  - Host directory is specified explicitly
  - Example:

    ```
    docker run -v /data/mongo:/data/db mongo:7-jammy
    ```

  - Good for development and debugging
- **Named volume**:
  - Docker manages location
  - Example:

    ```
    docker volume create mongo-data
    docker run -v mongo-data:/data/db mongo:7-jammy
    ```

Docker Networks

- Docker creates a default **bridge network** for containers
- Common network types:
  - **bridge**: default, for containers on a single host
  - **host**: container shares host network namespace
  - **custom bridge**: user-defined network with automatic DNS
- Example: create a custom network

```
docker network create app-net
docker run --network app-net --name db mongo:7-jammy
docker run --network app-net --name api my-api-image
```

Container Lifecycle

- Key stages:
  - **create**: container is defined but not running
  - **start**: container process starts
  - **stop**: container receives signal to stop
  - **remove**: container is deleted
- Common commands:

```
docker create ...
docker start my-container
docker stop my-container
docker rm my-container
```

Installing Docker & Working with the Docker CLI

- Docker Engine on **Linux, macOS, Windows**
- First test: run `hello-world`
- Essential Docker CLI commands

Installing Docker on Linux (Ubuntu Example)

- Add Docker's official GPG key and Add the repository to Apt sources:

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

Installing Docker on Linux (Ubuntu Example)

- Update package index and install Docker Engine:

```
sudo apt update
sudo apt install \
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

- After installation:

```
sudo systemctl status docker
```

Running Your First Container

- Test Docker with the official **hello-world** image:

```
sudo docker run hello-world
```

- What happens:
  - Docker checks if `hello-world` image exists locally
  - If not, it **pulls** it from Docker Hub
  - Runs a container that prints a welcome message and exits
- Confirms:
  - Engine, image download, and container runtime all work

Essential Container Management Commands

- List running containers:

```
docker ps
```

- List all containers (including stopped):

```
docker ps -a
```

- View logs:

```
docker logs my-container
```

- Execute a command inside a running container:

```
docker exec -it my-container bash
```

Working with Images

- Pull an image from a registry:

```
docker pull mongo:7-jammy
```

- List local images:

```
docker images
```

- Remove an image:

```
docker rmi mongo:7-jammy
```

- Push an image:

```
docker push mvmanh/my-api:1.0.0
```

Container Isolation

- **Filesystem isolation**:
  - Each container has its own root filesystem
- **Process isolation**:
  - Processes inside container see only their own environment
- **Network isolation**:
  - Each container has its own virtual network interface
- Benefit:
  - Safer, more predictable deployments
  - Easier debugging because environment is controlled

Running Simple Containers (MongoDB Example)

- Example command:

```
docker run --name mongo-demo -p 12345:27017 -d --rm mongo:7-jammy
```

- Purpose:
  - Start a MongoDB database for local development
  - Expose it on port **12345** on the host
- This is a realistic pattern used in backend and DevOps workflows

Breaking Down the MongoDB Command

- `--name mongo-demo`
  - Assigns a readable name to the container
- `-p 12345:27017`
  - Maps host port `12345` to container port `27017`
- `-d`
  - Runs container in **detached** mode (in the background)
- `--rm`
  - Automatically removes the container when it stops
- `mongo:7-jammy`
  - Image name and tag from Docker Hub

Inspecting Logs and Entering the Container

- View MongoDB logs:

```
docker logs mongo-demo
```

- Follow logs in real time:

```
docker logs -f mongo-demo
```

- Enter the container shell:

```
docker exec -it mongo-demo bash
```

- From the shell, you can:
  - Inspect `/data/db`
  - Use MongoDB shell (if installed in the image)

Stopping and Removing Containers

- Gracefully stop the container:

```
docker stop mongo-demo
```

- Because we used `--rm`, the container is removed automatically
- Without `--rm`, you can remove it manually:

```
docker rm mongo-demo
```

- Habit for DevOps engineers:
  - Regularly clean up unused containers and images to save disk space

Connecting External Apps to Containers

- Example: Node.js app connecting to MongoDB container
- From the host, connection string:

```
mongodb://localhost:12345/mydb
```

- From another container on the same network:
  - Use container name (`mongo-demo`) and internal port:

```
mongodb://mongo-demo:27017/mydb
```

- This pattern is common in:
  - Local development
  - Integration tests
  - Docker Compose setups

Dockerfile & Image Building

- What is a **Dockerfile**?
- Common instructions: `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, `CMD`, `ENTRYPOINT`
- Image layers, caching, and efficient builds

What Is a Dockerfile?

- A **Dockerfile** is a text file with instructions to build an image
- It defines:
  - Base image (OS + runtime)
  - Files to copy into the image
  - Commands to run during build
  - Default command when the container starts
- A Dockerfile is **infrastructure as code** for container images

Simple Node.js API Dockerfile

```
FROM node:20-alpine

WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production

COPY . .
EXPOSE 3000

ENV NODE_ENV=production
CMD ["node", "server.js"]
```

- Uses a small **Alpine** base image
- Installs dependencies, copies app code. Exposes port **3000**
- Sets the default command to run the API

Building and Running the Image

- Build the image:

```
docker build -t mvmanh/node-api:1.0.0 .
```

- Run the container:

```
docker run --name node-api-demo -p 3000:3000 -d mvmanh/node-api:1.0.0
```

- Test the API:

```
curl http://localhost:3000/health
```

Image Layers and Caching

- Each Dockerfile instruction creates a **layer**
- Docker reuses unchanged layers to speed up builds
- Best practices:
  - Put `COPY package*.json` and `RUN npm ci` before copying source code
  - This way, dependencies are cached when only code changes
- Example:
  - Change only `server.js` → Docker builds only the top layers

Using .dockerignore Effectively

- `.dockerignore` prevents unnecessary files from being copied into the image
- Example `.dockerignore`:

```
node_modules
npm-debug.log
.git
.env
tests
```

- Benefits:
  - Smaller images
  - Faster builds
  - Lower risk of copying secrets (e.g. `.env` files)

Environment Variable Best Practices (Inside Images)

- **Never bake secrets** into images:
  - API keys
  - Database passwords
  - JWT secrets
- Use **environment variables** at runtime instead:
  - Pass them with `-e` flag or via Compose/Kubernetes
- Treat images as **public**, even if stored in private registries:
  - Assume that images might leak
  - Minimize sensitive data in layers

ARG vs ENV in Dockerfiles

- `ARG`:
  - Build-time variable
  - Not available at container runtime
- `ENV`:
  - Runtime environment variable
  - Visible inside the container
- Example:

```
ARG APP_VERSION=1.0.0
ENV NODE_ENV=production
LABEL app.version=${APP_VERSION}
```

- Do **not** pass secrets via `ARG` or `ENV` in Dockerfile

Minimal Base Images and Multi-Stage Builds

- Use **minimal base images**:
  - `alpine`, `distroless`, or language-specific slim images
- Multi-stage builds:
  - First stage: build the app (compiler, tools)
  - Second stage: copy only the build artifacts into a small runtime image

Minimal Base Images and Multi-Stage Builds

- Example (preview):

```
FROM node:20-alpine AS build

WORKDIR /usr/src/app
COPY . .
RUN npm ci && npm run build

FROM node:20-alpine

WORKDIR /usr/src/app
COPY --from=build /usr/src/app/dist ./dist
CMD ["node", "dist/server.js"]
```

Working with Registries (Public & Private)

- Docker Hub for public and small private projects
- AWS ECR for production-grade private images
- Tagging, logging in, pushing and pulling images
- Basic container security practices

Docker Hub Overview

- Public registry at **hub.docker.com**
- You can create:
  - Public repositories (free)
  - Private repositories (limited free, paid tiers)
- Typical workflow:

```
docker login
docker tag node-api:1.0.0 mvmanh/node-api:1.0.0
docker push mvmanh/node-api:1.0.0
```

- Other machines can then:

```
docker pull mvmanh/node-api:1.0.0
```

AWS ECR Overview

- **Elastic Container Registry (ECR)**: AWS-managed private registry
- Common steps:
  - Create ECR repository in AWS Console
  - Authenticate Docker with AWS:

```
aws ecr get-login-password --region ap-southeast-1 | \
docker login \
--username AWS \
--password-stdin \
123456789012.dkr.ecr.ap-southeast-1.amazonaws.com
```

AWS ECR Overview

- Tag and push:

```
docker tag node-api:1.0.0 \
123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/node-api:1.0.0

docker push \
123456789012.dkr.ecr.ap-southeast-1.amazonaws.com/node-api:1.0.0
```

- `123456789012.dkr.ecr.ap-southeast-1.amazonaws.com` is your AWS account's ECR registry URL

Image Tagging and Versioning

- Use meaningful tags:
  - Semantic versioning: `1.0.0`, `1.1.3`
  - Environment: `staging`, `production`
  - Git commit hash: `node-api:1.0.0-abc1234`
- Avoid relying only on `latest`:
  - Hard to know which code is running
  - Difficult to roll back
- Good practice:
  - Tag with both version and commit hash

Container Security Basics

- Use **trusted base images**:
  - Official images from Docker Hub
  - Verified publisher images
- Scan images for vulnerabilities (e.g. **Trivy**):

```
trivy image mvmanh/node-api:1.0.0
```

- Avoid running as **root** inside the container:
  - Create a non-root user in Dockerfile
- Keep images up to date:
  - Regularly rebuild with patched base images

Local Image Management vs Remote Registries

- Cleaning up local images and containers
- When to store images locally
- When to push images to registries

Cleaning Up Local Resources

- Remove stopped containers:

```
docker container prune
```

- Remove unused images:

```
docker image prune
```

- Remove everything unused:

```
docker system prune
```

- Good habit:
  - Run cleanup periodically on development machines and CI runners

Local Images vs Remote Registries

- **Local images**:
  - Fast to run on your own machine
  - Not automatically available to your team
- **Remote registries**:
  - Central place for images
  - Required for CI/CD and production deployments
- Workflow:
  - Build locally or in CI
  - Push to registry
  - Deploy from registry

Private Images and Production Storage

- Use private repos for:
  - Internal services
  - Backend APIs with proprietary code
- Options:
  - Docker Hub private repositories
  - AWS ECR, GCP Artifact Registry, Azure Container Registry
- For production:
  - Prefer cloud-native registries (ECR, etc.) near your cluster/VMs
  - Reduces latency and improves reliability

Developer Workflow with Docker (CLI Overview)

- Typical lifecycle:
  - **Build → Run → Test → Tag → Push → Deploy**
- Important commands:
  - `docker build`, `docker run`, `docker stop`, `docker rm`
  - `docker logs`, `docker inspect`, `docker network ls`

Typical Backend Developer Workflow

- After implementing a new feature:

```
git pull
docker build -t node-api:1.1.0 .
docker run --name node-api-dev -p 3000:3000 -d node-api:1.1.0
```

- Test locally:

```
curl http://localhost:3000/api/products
```

- If tests pass:
  - Tag and push to registry
  - Create or update deployment manifest (Compose, Kubernetes, etc.)

Inspecting Containers and Networks

- Inspect container details:

```
docker inspect node-api-dev
```

- List networks:

```
docker network ls
```

- Inspect network:

```
docker network inspect app-net
```

- DevOps use cases:
  - Debug connectivity issues
  - Verify environment variables. Check attached volumes

Bridge Networks vs Custom Networks

- Default **bridge** network:
  - All containers use it if no network is specified
  - Container names may not resolve as DNS
- **Custom bridge** network:
  - Automatic DNS by container name
  - Better isolation per project

```
docker network create shop-net
docker run --network shop-net --name db mongo:7-jammy
docker run --network shop-net --name api mvmanh/node-api:1.0.0
```

- Now `api` can use `mongodb://db:27017/shop` as connection string

Docker Compose for Multi-Service Applications

- Why **Docker Compose**?
- Structure of `docker-compose.yml`
- Services, networks, volumes
- Key commands: `docker compose up`, `down`, `logs`

Why We Need Docker Compose

- Real applications rarely consist of a single container:
  - Backend API
  - Database, Cache
  - Admin UI (e.g. Mongo Express)
- Managing multiple `docker run` commands is:
  - Hard to reproduce
  - Error-prone
- **Docker Compose**:
  - Describes multi-container apps as YAML
  - One command to start/stop the entire stack

Simple Compose File: Node API + MongoDB

```yaml
version: "3.9"
services:
  db:
    image: mongo:7-jammy
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
  api:
    build: ./api
    ports:
      - "3000:3000"
    environment:
      - MONGODB_URI=mongodb://db:27017/shop
volumes:
  mongo-data:
```

Simple Compose File: Node API + MongoDB

- Docker compose yaml file above:

```yaml
version: "3.9"
services:
  db:
    ...
  api:
    ...
volumes:
  mongo-data:
```

- Two services share default network
- `api` connects to `db` using service name (e.g. `mongodb://db:27017/shop`)

Managing Services with Docker Compose

- Start all services in the background:

```
docker compose up -d
```

- View logs:

```
docker compose logs -f
```

- List running services:

```
docker compose ps
```

- Stop and remove services:

```
docker compose down
```

Rebuilding Images with Docker Compose

- When code changes in the `api` service:

```
docker compose build api
docker compose up -d api
```

- Or rebuild and restart everything:

```
docker compose up -d --build
```

- Compose makes it easy to:
  - Recreate services
  - Rotate configurations
  - Apply updates consistently

Environment Variables in Compose

- Use `.env` file for non-sensitive configs:

```
NODE_ENV=development
API_PORT=3000
```

Environment Variables in Compose

- In `docker-compose.yml`:

```yaml
services:
  api:
    build: ./api
    ports:
      - "${API_PORT}:3000"
    env_file:
      - .env
    environment:
      - NODE_ENV=${NODE_ENV}
```

- Important:
  - Do **not** commit secrets in `.env` to Git
  - Use secret managers for passwords and keys

Preview: Secret Management

- `.env` files are convenient but risky for secrets:
  - Can be accidentally committed
  - Often copied to other environments
- Better approaches (preview for later modules):
  - Docker Swarm/Kubernetes secrets
  - AWS SSM Parameter Store
  - AWS Secrets Manager
- Principle:
  - **Store secrets outside images and code**
  - Inject them at runtime via secure channels

Hands-On Labs Overview

- Lab 1: Dockerize a Node.js Express API
- Lab 2: Multi-service app with Docker Compose
- Lab 3: Manual scaling without load balancing

Lab 1: Dockerize a Single Node.js Express API

- Tasks:
  - Implement a simple Express API (e.g. `/health`, `/products`)
  - Write a Dockerfile:
    - Use official Node base image
    - Install dependencies with `npm ci`
    - Set `NODE_ENV` via environment variable
  - Build and run:

```
docker build -t your-name/node-api:lab1 .
docker run -p 3000:3000 -d your-name/node-api:lab1
```

  - Push image to Docker Hub

Lab 2: Multi-Service App with Docker Compose

- Compose stack:
  - `db`: MongoDB
  - `mongo-express`: Web UI for MongoDB
  - `api`: Node.js Express Web API
- Key skills:
  - Define services, networks, and volumes
  - Use environment variables for: MongoDB URI, credentials for `mongo-express`
- Test:
  - API via `curl` or browser
  - Database via `mongo-express` in browser

Lab 3: Manual Scaling (No Load Balancer Yet)

- Extend Lab 2:
  - Build image for `api`
  - Run multiple instances:

```
docker run -p 3001:3000 -d your-name/node-api:lab2
docker run -p 3002:3000 -d your-name/node-api:lab2
docker run -p 3003:3000 -d your-name/node-api:lab2
```

- Observe:
  - Multiple API processes on different ports
  - CPU and memory usage with `docker stats`
- Motivation:
  - Understand scaling before introducing load balancers (next module)

Homework

- Install Docker on an Ubuntu server (e.g. AWS EC2)
- Upload a small project with `docker-compose.yml`
- Use environment variables correctly:
  - No secrets in images
  - Use `.env` only for non-sensitive configs
- Run the project in **detached** mode:

```
docker compose up -d
```

- Verify:
  - Logs, Volumes and data persistence
  - Containers restart correctly after server reboot